UMHackathon 2026 • Domain 1

RefundFlow AI

Multi-platform customer refund workflow assistant powered by Z.AI GLM

Transforms messy refund messages into structured cases, validation decisions, specialist agent tasks, and prototype approval paths.

Team Name: Quad Hack
Members: Jeremy, Joshua, Tan Kwan Hong, Kong Yih Liang
Prototype: FastAPI backend + HTML frontend

Preliminary Round Pitch Deck

Refunds are fragmented, manual and confusing

Across buying platforms, customers and support teams repeat the same interpretation work.

Different platform rules

Shopee, Lazada, TikTok Shop, web stores and subscriptions each require different refund steps, evidence and timing.

Messy customer messages

Users often submit incomplete, emotional or ambiguous messages without order ID, product name, evidence or contact details.

Slow support triage

Support teams manually read, ask follow-up questions, check orders, create cases and decide escalation paths.

1. **Customer complains** — unstructured
2. **Support interprets** — manual
3. **System lookup** — platform-specific
4. **Decision / refund** — risk-sensitive

Core opportunity: use GLM to coordinate the workflow, not merely generate a reply.

Target users and user stories

Designed around both customer experience and back-office refund operations.

Online Shopper

“I want a simple refund path even if I do not know the exact policy.”

Support Agent

“I want the system to extract details and ask the right follow-up questions.”

Ops Admin

“I want risky or high-value refunds routed to manual review.”

Small Seller

“I want consistent case records instead of scattered messages.”

Primary user story

As a customer, I want to submit a messy refund request and be guided to the correct next step, so I do not waste time guessing what information is required.

Operations user story

As a support team, I want structured refund cases, validation flags, and escalation decisions, so I can handle requests consistently and safely.

Market relevance: customer support automation + e-commerce operations

Solution: RefundFlow AI

A stateful refund workflow engine where Z.AI GLM is the central reasoning coordinator.

Customer request

messy text, missing fields, platform context

→

Z.AI GLM Reasoning Engine

intent • missing info • workflow stage • next action • structured JSON

→

Action mapping

ask, validate, create case, approve, reject, review

Missing information

Follow-up questions

Refund case

Structured case record

Validation result

Policy + order flags

Manual review

High-risk escalation

Why it fits Domain 1: GLM coordinates multi-step workflow decisions and tool/API interactions, beyond simple chatbot replies.

MVP scope for the preliminary round

Focused on a demonstrable, testable refund orchestration flow.

Built MVP features

1. **Refund intake**
   Accepts messy customer refund request text.
2. **Missing info detection**
   Identifies required fields such as order ID, reason, product and contact.
3. **Follow-up workflow**
   Collects additional answers and updates the workflow state.
4. **Order validation**
   Checks against prototype order database and policy flags.
5. **Decision routing**
   Creates case, approves prototype, rejects, or routes manual review.

Out of scope now

- Real money movement or payment gateway execution
- Real Shopee/Lazada/TikTok API integration
- Long-term production database and authentication

Success criteria

- Demo handles one full refund flow end-to-end
- Incomplete request is blocked and clarified
- High-risk refund routes to human review

Prototype demo flow

This is the exact flow to show in the 10-minute pitching video.

1

Start backend

uvicorn customer_refund_backend:app --port 8000

2

Open frontend

customer_refund_app_realistic.html

3

Submit request

Wrong item / damaged item / duplicate payment

4

GLM analyzes

intent, stage, missing info, next action

5

Execute outcome

case created, validation, manual review or approval

Recommended live demo input

“I want a refund because I received the wrong item. My order ID is ORD12345 and my email is customer@email.com. I paid by card.”

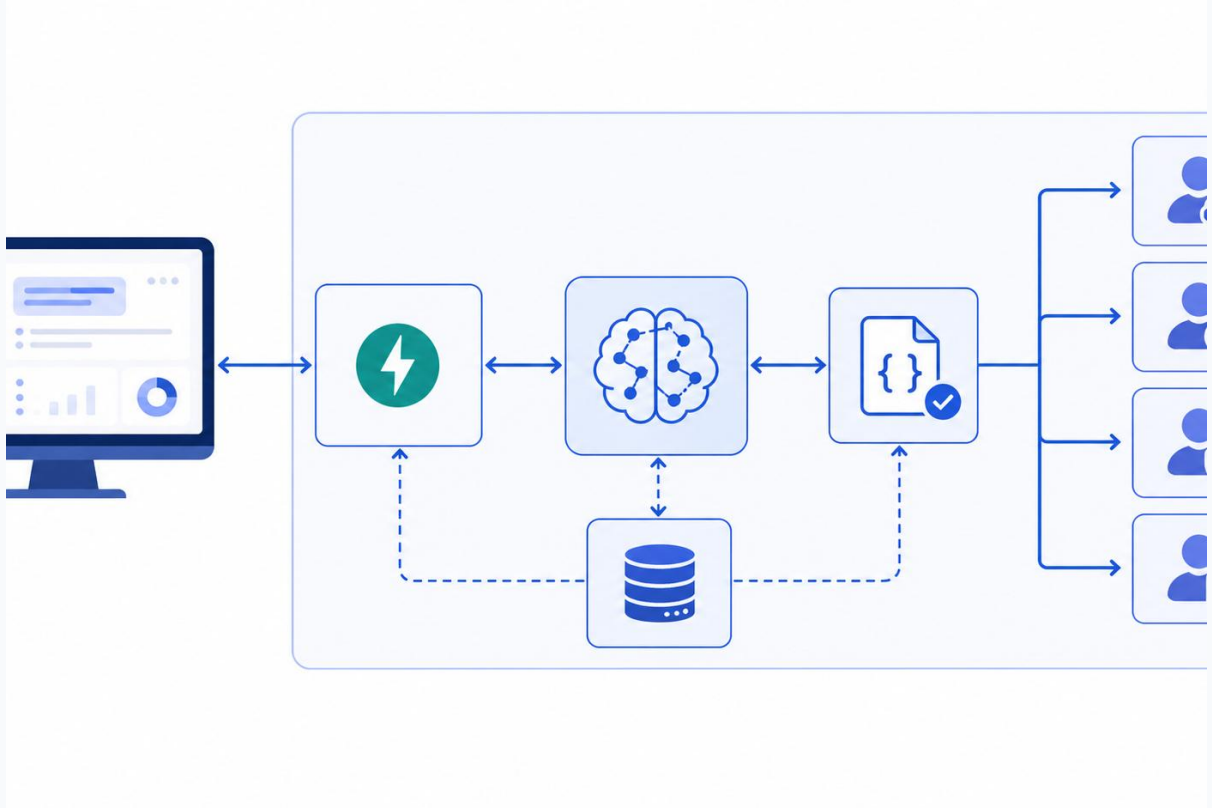
Show these proof points

Intent • Missing information • Workflow stage • Agent tasks • Raw JSON • Admin case view

System architecture

Frontend, FastAPI backend, GLM reasoning layer, validation rules and specialist refund agents.

Frontend	HTML/CSS/JS customer + admin view
Backend API	FastAPI, Pydantic, CORS, REST routes
GLM Service Layer	Z.AI GLM prompt + JSON response parsing
Validation + Action Map	Required fields, mock order DB, safety rules
Agent Tasks	intake, eligibility, case, approval, risk review



Core API routes: /refund/start • /refund/follow-up • /refund/execute/{workflow_id} • /refund-cases • /mock-orders • /health

Backend orchestration and specialist agents

The backend converts one GLM decision into executable workflow actions.

Allowed next_action values

ask_follow_up_questions	→ Customer Information Agent
generate_refund_checklist	→ Checklist Agent
create_refund_case	→ Case Creation Agent
validate_refund_request	→ Eligibility Agent
calculate_refund_estimate	→ Refund Estimation Agent
approve_refund_prototype	→ Approval Agent
reject_refund_request	→ Rejection Agent
manual_review_required	→ Risk Review Agent

Structured GLM JSON contract

intent • confidence • workflow_stage • missing_information • next_action • structured_output

Parser rejects invalid JSON and falls back to manual_review_required.

Backend safety net

- Cannot approve when required information is missing
- High-value / risky keywords route to human review
- Mock order database verifies known order IDs and contacts

Data model and stateful workflow

Prototype uses in-memory data stores to prove orchestration before real platform integration.

workflow_db

workflow_id, status, intent, stage, missing info, agent tasks, timestamps

refund_case_db

refund_case_id, workflow_id, details, eligibility, status, priority

mock_order_db

order_id, contact, product, payment status, amount, risk level

Input constraints

- Maximum input size: 6,000 characters
- Very short input is rejected with customer-friendly message
- Invalid or unusable GLM output falls back to manual review

Validation checks

- Required fields: order ID, reason, product/service, contact
- Order not found or contact mismatch blocks approval
- High-risk order routes to manual review

Quality assurance and testing plan

QATD follows the sample: traceability, risks, environment, test cases, AI boundary tests.

Core test cases

TC-01 Happy path	Valid order ORD12345 → eligible refund workflow
TC-02 Missing info	No order ID → follow-up questions, no approval
TC-03 Contact mismatch	Wrong customer email → validation blocked
TC-04 High-risk order	ORD77777 → manual review required
TC-05 NFR/API	/health response + backend endpoint reliability
AI-01 JSON contract	GLM must return valid structured JSON only

How to test the code

1) Run backend: uvicorn customer_refund_backend:app --reload --port 8000
2) Open frontend with Python HTTP server
3) Run automated tests: pytest -q

Risk mitigation

- GLM hallucination → JSON parser + allowed action whitelist
- Unsafe refund approval → backend required-field enforcement
- Real payment risk → prototype-only approval disclaimer

Judging criteria coverage

The deck, documents and demo are aligned to preliminary judging requirements.

Product thinking	Clear refund pain point, target users, original workflow value, focused MVP scope
Architecture	Frontend + FastAPI + GLM service layer + validation + agent orchestration
Engineering	Actual code repository, modular functions, API routes, structured data, demo-ready UI
Quality assurance	Risk matrix, traceability, API tests, AI output tests, edge cases and NFRs
Communication	10-minute video plan: slides + live demo + code walkthrough + testing proof

Submission package: PRD PDF • SAD PDF • QATD PDF • Code repository • Pitch deck • 10-minute pitch video with prototype demo

Roadmap

From prototype to real refund operations

RefundFlow AI proves the workflow layer first. Next, it can connect to real platform APIs, OCR receipt extraction, persistent databases and human approval dashboards.

Platform APIs

Real order/refund status integration

Document AI

Receipt and chat screenshot extraction

Policy engine

Platform-specific refund rule knowledge base

Deployment

Cloud hosting, database, authentication, monitoring

Thank you

Q&A